FORMATO DE DESARROLLO E INTEGRACIONES

FT-SL-011-APIs
Portal clientes B2C

STF GROUP S.A.
INFORMATICA – Py. CREDI10 SOFTWARE PROPIO

Cali, jueves 19 de febrero del 2024

TABLA DE CONTENIDO

INTRODUCCION 3
1. DETALLE TECNICO 4
1.1 Detallar componentes existentes..... 4
1.2 Detallar funcionalidades 5
1.3 Observaciones y/o Comentarios..... 15
2. CASOS DE PRUEBAS UNITARIAS 16

INTRODUCCION

Este documento muestra en detalle todas las actividades de la solución a desarrollar con sus dependencias, entradas y salidas. Así como los casos de pruebas unitarias realizadas a la solución.

NOTA: El desarrollo de cualquier solución de requerimiento debe seguir los estándares de desarrollo que define Arquitectura.

1. DETALLE TECNICO

1.1 Descripción del Requerimiento:

1.1.1 Requerimiento

Disponer API para suministrar diferente información al portal de clientes credi10 (B2C).

A continuación, se detalla los procesos importantes para cumplir con el requerimiento.

- Crear endpoint para consultar información general del cliente.
- Crear endpoint para consultar los prestamos vigentes y cancelados del cliente.
- Crear endpoint para consultar los pagos realizados del cliente.

1.2 Detallar componentes existentes

Para el desarrollo de los requerimientos se cuenta con 1 solución desarrollada en .NET Framework 4.8 llamada “Créditos”.

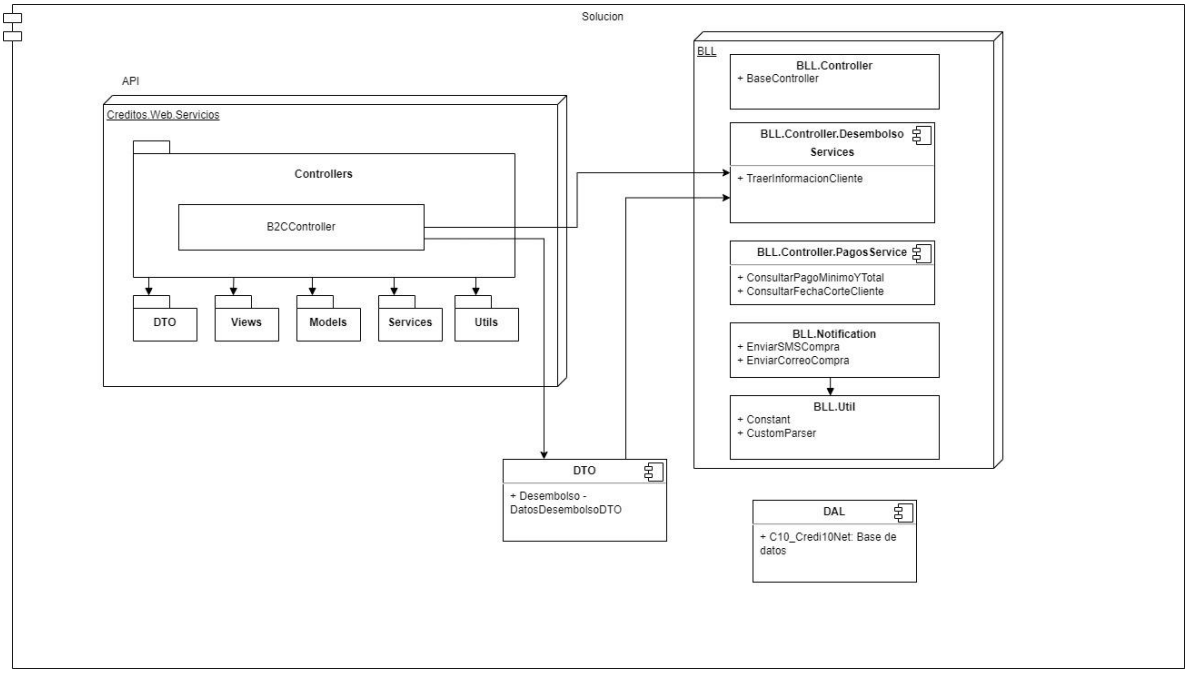
Se realiza creación del proyecto web API “*Creditos.Web.Servicios*” donde se establece el funcionamiento de los requerimientos correspondientes a Link de recaudo, botón ecommerce e integración B2C.

Nombre del Componente	Descripción
SOLUCION	
Creditos.Web.Servicios	API CheckoutController
Creditos.Web.Admin	Controladores y vistas implementadas para cumplir con el propósito de informar el valor del pago y recibir el valor que quieran aplicar.
Creditos.BLL	Proyecto que contiene las clases y métodos que implementan la lógica de negocio de la aplicación, esta biblioteca de clases será el encargado de calcular los pagos, procesarlos, generar y enviar el ticket. Este proyecto se integra con el proyecto Web.Admin, el proyecto DAL y el proyecto DTO.
Creditos.DAL	Proyecto encargado de tener el acceso y la interacción a la base de datos C10_Credi10NET de la solución.
Creditos.DTO	Proyecto que contiene los diferentes modelos usados en toda la aplicación.

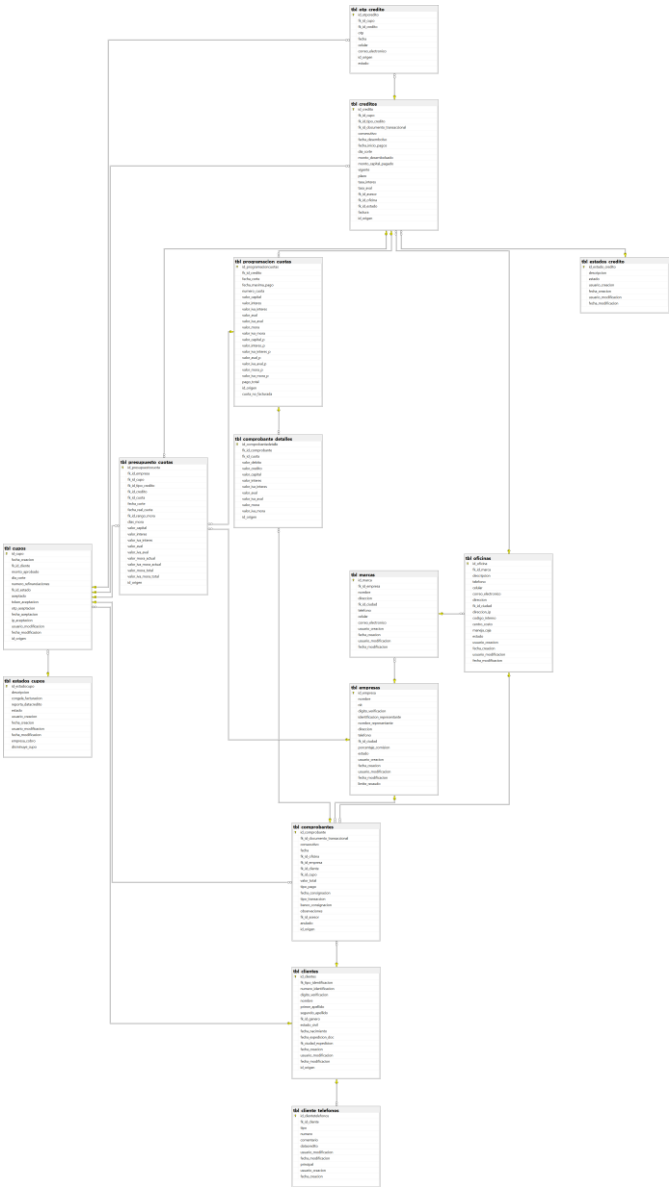
Rotativa	Herramienta de línea de comandos para representar HTML en PDF, encargado de crear los tickets de impresión del pago.
BASE DE DATOS	
C10_Credi10NET	Tablas y Procedimientos almacenados

1.3 Detallar funcionalidades

1.3.1 Diagrama de Componentes



1.3.2 MER



Parametrizaciones

Descripción	Grupo	Valor
Link conexión consumo API pagos	Link	https://apicredinet.studiof.com.co:32448/payment/paymentLink

1.3.3 Proceso

Se implementa el componente B2CController encargado de recibir 4 tipos de peticiones diferenciadas por el atributo “operación”. Para este proceso se establece el valor “consultar”

consiste en obtener la identificación del cliente y generar la información del cliente existente en la base de datos con atributos específicos.
Para esto se necesita recibir un objeto JSON en el cuerpo de la solicitud POST como el que se observa a continuación:

```
{"datos":"CoIqISGDjiQV5NB\/CPmMb+9JUwlz\/4Ygt9eF+9MwoVoRkOk
    +qixQKDwEfnujHO0Zt9FjU2thwI71z0Gu6rOg3urmhFovY7jN9rl+lo+WvUPGjuhqB4wd3lQQSKsJZATXz0oBgsKn3A\/
    9hTnsPEtC1b2VwXkuMSsSFLKt7TZpoVNI6Q4B1YSnzyd94RiO6bbNXZm3FlRBhZ4pXm0exEBAoB58rXA3ud3mOSfdPK+yzPE01
    +M\/5Lvfj+HY44fCiVTuM2IZ+L6y9Lnp6VQsh026n4GwxQGo0ykQbZIYT6fYKtGiVENnAqyTaETMfXbwxyayIK76Y1mPm\/
    CAMXe6Jz++Y1Gys7\/xFvi4m7BIs0SN1wE="}
```

Donde el contenido del atributo “datos” corresponde a una encriptación del siguiente objeto

```
{
“operacion”: “consultar”,
“identificacion”: “16736984”,
“usuario”: “xxx”,
“password”: “xxx”
}
```

```
public class B2CController : Controller
{
    // POST: B2C/Create
    [HttpPost]
    0 referencias
    public async Task<ActionResult> b2c(CheckoutDto checkoutDto)
    {
        DesembolsosServices desembolso = new DesembolsosServices();
        PagosService pagosService = new PagosService();
        UserServices userServices = new UserServices();
        EncryptionAES encryptionAES = new EncryptionAES();

        object response = new object();
        object dataObj = new object();
        bool hasError = false;
        int codeOk = 200;
        int codeError = 405;
        string messageOk = "Registros encontrados!";
        string messageError = "";

        try
        {
            // Desencriptar
            string valueEncrypted = JsonConvert.SerializeObject(checkoutDto.datos);
            var decryptedData = encryptionAES.Decrypt(valueEncrypted.Replace("\"", ""));

            DataCheckoutDto dataCheckoutDto = JsonConvert.DeserializeObject<DataCheckoutDto>(decryptedData);

            bool userChecked = userServices.ValidateUser(dataCheckoutDto.Usuario, dataCheckoutDto.Password);
            if (!userChecked)
            {
                hasError = true;
                messageError = "No autorizado";
                dataObj = new
                {
                    status = "error",
                    data = new object(),
                    code = codeError,
                    mensaje = messageError
                };

            } else
            {
                var sucursal = ConfigurationManager.AppSettings["Sucursal"];

                if (dataCheckoutDto.Operacion.ToLower() == "consultar")...
                else if (dataCheckoutDto.Operacion.ToLower() == "consultarliquidacion")...
                else if (dataCheckoutDto.Operacion.ToLower() == "consultarprestamos")...
                else if (dataCheckoutDto.Operacion.ToLower() == "consultarpagos")...
                else...
            }
        }
        catch (Exception ex)
        {
            hasError = true;
            messageError = ex.Message;
            dataObj = new
            {
                status = "error",
                data = new object(),
                code = codeError,
                mensaje = messageError
            };
        }

        var encryptedData = encryptionAES.Encrypt(JsonConvert.SerializeObject(dataObj));
        response = new
        {
            status = hasError ? "error" : "success",
            datos = encryptedData,
            code = hasError ? codeError : codeOk,
            mensaje = hasError ? messageError : messageOk
        };

        return Json(response);
    }
}
```

Al realizar cualquier solicitud a la API de B2CController entrará al método asíncrono llamado b2c, que procesará todas las solicitudes y, dependiendo de la operación recibida en el cuerpo de la petición, hacer una u otra tarea.

```
// Desencriptar
string valueEncrypted = JsonConvert.SerializeObject(checkoutDto.datos);
var decryptedData = encryptionAES.Decrypt(valueEncrypted.Replace("\"", ""));

DataCheckoutDto dataCheckoutDto = JsonConvert.DeserializeObject<DataCheckoutDto>(decryptedData);

bool userChecked = userServices.ValidateUser(dataCheckoutDto.Usuario, dataCheckoutDto.Password);
if (!userChecked)
{
    hasError = true;
    messageError = "No autorizado";
    dataObj = new
    {
        status = "error",
        data = new object(),
        code = codeError,
        mensaje = messageError
    };

} else...
```

El objeto json recibido en la petición se desencripta con claves específicas y se inicia con el proceso validación del usuario y password suministrados en el objeto, si no son correctas las credenciales el sistema retornará “No Autorizado”, en caso de ser correctas, se obtendrá la información necesaria para devolver el objeto especificado en la API.

```
if (dataCheckoutDto.Operacion.ToLower() == "consultar")
{
    using (var dbContext = new BaseContext())
    {
        var idSucursal = int.Parse(sucursal);
        var infoSucursal = dbContext.tbl_oficinas.FirstOrDefault(x => x.id_oficina == idSucursal);
        var infoClient = desembolso.TraerInformacionCliente(dataCheckoutDto.Identificacion, infoSucursal.codigo_interno, false);

        if (infoClient.Data != null)
        {
            DatosDesembolsoDTO datosDesembolsoDTO = JsonConvert.DeserializeObject<DatosDesembolsoDTO>(infoClient.Data);

            var cliente = dbContext.tbl_clientes.FirstOrDefault(x => x.numero_identificacion == datosDesembolsoDTO.NumeroCedula);

            var cupo = dbContext.tbl_cupos.Where(x => x.fk_id_cliente == cliente.id_clientes).First();
            var estados = dbContext.tbl_estados_cupos.Where(x => x.id_estadocupo == cupo.fk_id_estado).First();
            decimal? balance = (datosDesembolsoDTO.CupoAprobado * (decimal)0.1) - datosDesembolsoDTO.ValorDisponible;

            dataObj = new
            {
                status = "success",
                data = new
                {
                    idcupo = cupo.id_cupo,
                    identificacion = datosDesembolsoDTO.NumeroCedula,
                    nombreclente = datosDesembolsoDTO.Nombre,
                    correo = datosDesembolsoDTO.CorreoElectronico,
                    celular = datosDesembolsoDTO.Celular,
                    descripcion = estados.descripcion,
                    aceptado = cupo.aceptado ? "Si" : "No",
                    diacorte = datosDesembolsoDTO.DiaCorte,
                    totalcupo = datosDesembolsoDTO.CupoAprobado,
                    saldo = balance,
                    disponible = datosDesembolsoDTO.ValorDisponible,
                    comentario = estados.descripcion.ToLower() == "Vigente".ToLower() ? "Activo" : "Inactivo"
                },
                code = codeOk,
                mensaje = messageOk
            };

        } else
        {
            hasError = true;
            messageError = infoClient.Message;
            dataObj = new
            {
                status = "error",
                data = new object(),
                code = codeError,
                mensaje = messageError
            };
        }
    }
}
```

La operación “consultar” tendrá como objetivo retornar los datos de un cliente en concreto. La información que obtendrá será idCupo, el número de identificación, el nombre completo, el correo y celular registrados como principales en credi10, el estado actual del cliente, su

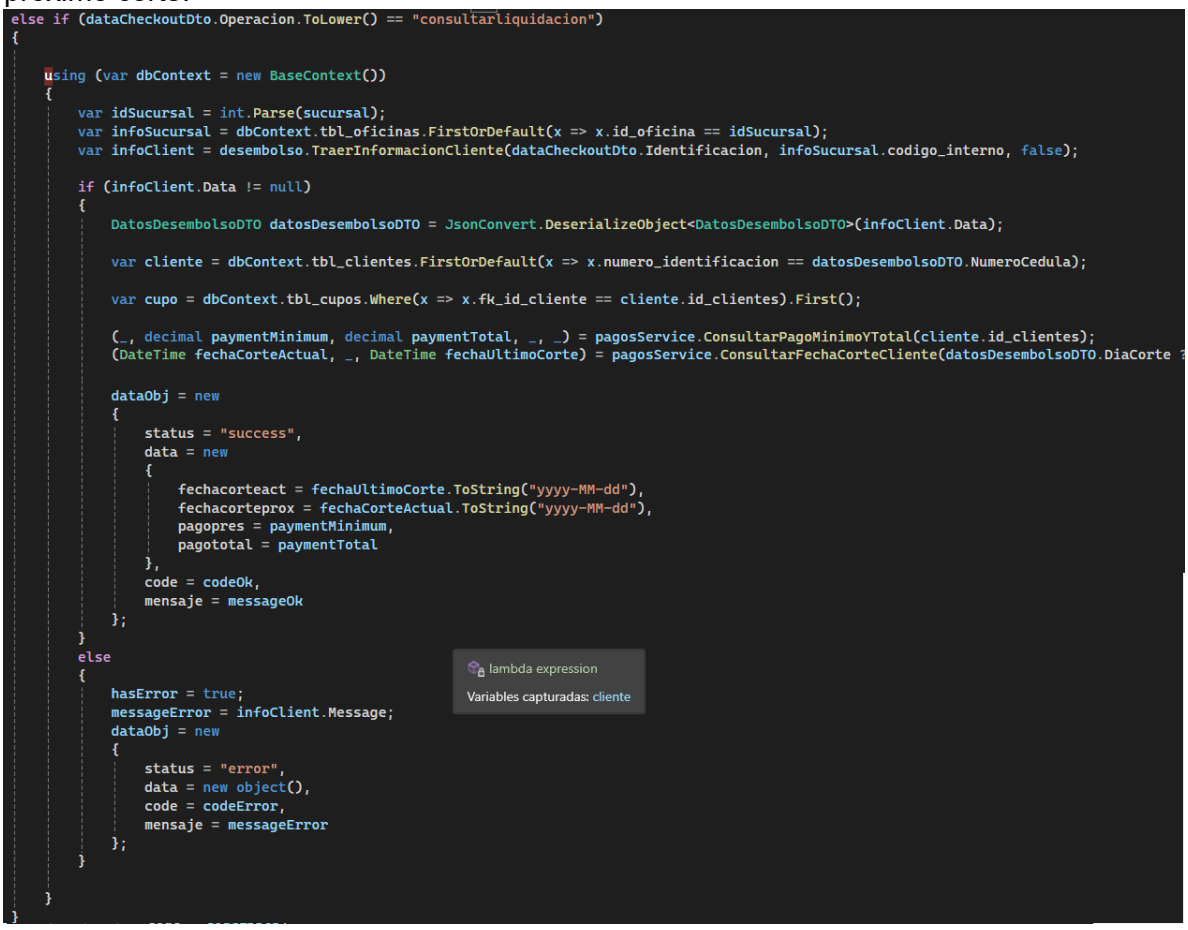
día de corte, el saldo disponible y el total del cupo. Todo el objeto que contiene la información será encriptado antes de ser enviado como respuesta de la petición.

- **Operación consultar liquidación**

Para este proceso se establece el valor “consultarliquidacion” que será enviado de manera encriptada en el mismo json descrito anteriormente. Donde el contenido del atributo “datos” corresponde a una encriptación del siguiente objeto

```
{
“operacion”: “consultarliquidacion”,
“identificacion”: “16736984”,
“usuario”: “xxx”,
“password”: “xxx”
}
```

La operación “consultarliquidacion” tendrá como objetivo retornar los datos de un cliente en concreto de la fecha corte que tiene actualmente con credi10, cual es la fecha máxima de pago, el pago minimo y total con los que cuenta a la fecha y cuando será la fecha de su próximo corte.



```
else if (dataCheckoutDto.Operacion.ToLower() == "consultarliquidacion")
{

    using (var dbContext = new BaseContext())
    {
        var idSucursal = int.Parse(sucursal);
        var infoSucursal = dbContext.tbl_oficinas.FirstOrDefault(x => x.id_oficina == idSucursal);
        var infoClient = desembolso.TraerInformacionCliente(dataCheckoutDto.Identificacion, infoSucursal.codigo_interno, false);

        if (infoClient.Data != null)
        {
            DatosDesembolsoDTO datosDesembolsoDTO = JsonConvert.DeserializeObject<DatosDesembolsoDTO>(infoClient.Data);

            var cliente = dbContext.tbl_clientes.FirstOrDefault(x => x.numero_identificacion == datosDesembolsoDTO.NumeroCedula);

            var cupo = dbContext.tbl_cupos.Where(x => x.fk_id_cliente == cliente.id_clientes).First();

            (_, decimal paymentMinimum, decimal paymentTotal, _, _) = pagosService.ConsultarPagoMinimoYTotal(cliente.id_clientes);
            (DateTime fechaCorteActual, _, DateTime fechaUltimoCorte) = pagosService.ConsultarFechaCorteCliente(datosDesembolsoDTO.DiaCorte

            dataObj = new
            {
                status = "success",
                data = new
                {
                    fechacorteact = fechaUltimoCorte.ToString("yyyy-MM-dd"),
                    fechacorteprox = fechaCorteActual.ToString("yyyy-MM-dd"),
                    pagopres = paymentMinimum,
                    pagototal = paymentTotal
                },
                code = codeOk,
                mensaje = messageOk
            };
        }
        else
        {
            hasError = true;
            messageError = infoClient.Message;
            dataObj = new
            {
                status = "error",
                data = new object(),
                code = codeError,
                mensaje = messageError
            };
        }

    }
}
```

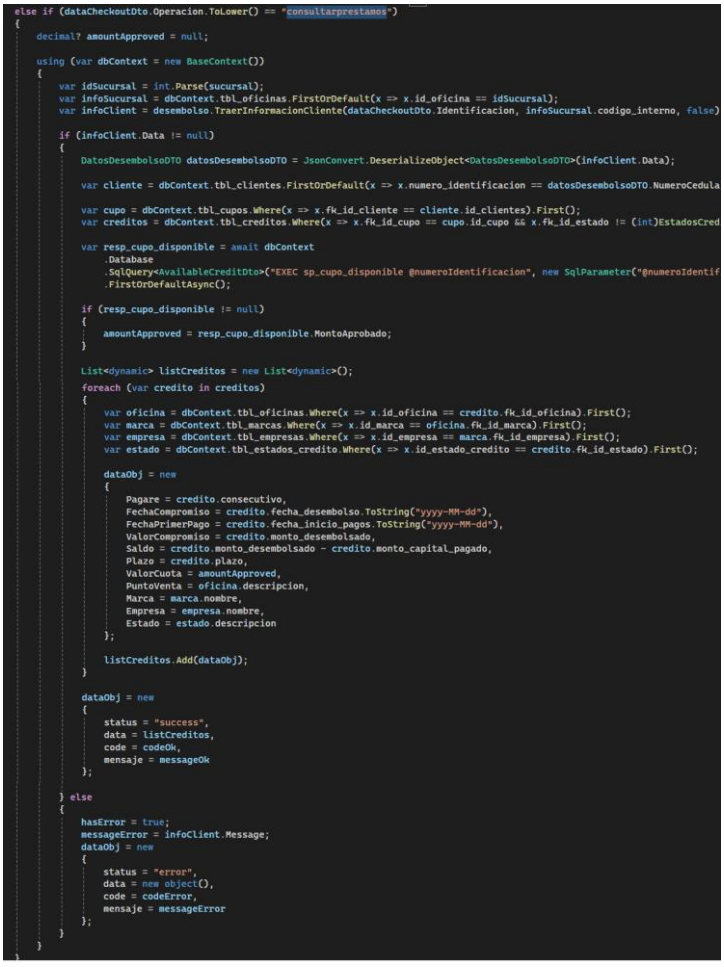
lambda expression
Variables capturadas: cliente

- **Operación consultar prestamos**

Para este proceso se establece el valor “consultarprestamos” que será enviado de manera encriptada en el mismo json descrito anteriormente. Donde el contenido del atributo “datos” corresponde a una encriptación del siguiente objeto:

```
{
“operacion”: “consultarprestamos”,
“identificacion”: “16736984”,
“usuario”: “xxx”,
“password”: “xxx”
}
```

La operación “consultarprestamos” tendrá como objetivo retornar los datos de un cliente en concreto de los créditos que tiene a la fecha históricamente y el estado de este, es decir si esta cancelado o si el crédito actualmente se encuentra vigente.



```
else if (dataCheckoutDto.Operacion.ToLower() == "consultarprestamos")
{
    decimal? amountApproved = null;

    using (var dbContext = new BaseContext())
    {
        var idSucursal = int.Parse(sucursal);
        var infoSucursal = dbContext.tbl_oficinas.FirstOrDefault(x => x.id_oficina == idSucursal);
        var infoClient = desembolso.TraerInformacionCliente(dataCheckoutDto.Identificacion, infoSucursal.codigo_interno, false)

        if (infoClient.Data != null)
        {
            DatosDesembolsoDTO datosDesembolsoDTO = JsonConvert.DeserializeObject<DatosDesembolsoDTO>(infoClient.Data);

            var cliente = dbContext.tbl_clientes.FirstOrDefault(x => x.numero_identificacion == datosDesembolsoDTO.NumeroCedula

            var cupo = dbContext.tbl_cupos.Where(x => x.fk_id_cliente == cliente.id_clientes).First();
            var creditos = dbContext.tbl_creditos.Where(x => x.fk_id_cupo == cupo.id_cupo && x.fk_id_estado != (int)EstadosCred

            var resp_cupo_disponible = await dbContext
                .Database
                .SqlQuery<AvailableCreditDto>("EXEC sp_cupo_disponible @numeroIdentificacion", new SqlParameter("@numeroIdentif
                .FirstOrDefaultAsync();

            if (resp_cupo_disponible != null)
            {
                amountApproved = resp_cupo_disponible.MontoAprobado;
            }

            List<dynamic> listCreditos = new List<dynamic>();
            foreach (var credito in creditos)
            {
                var oficina = dbContext.tbl_oficinas.Where(x => x.id_oficina == credito.fk_id_oficina).First();
                var marca = dbContext.tbl_marcas.Where(x => x.id_marca == oficina.fk_id_marca).First();
                var empresa = dbContext.tbl_empresas.Where(x => x.id_empresa == marca.fk_id_empresa).First();
                var estado = dbContext.tbl_estados_credito.Where(x => x.id_estado_credito == credito.fk_id_estado).First();

                dataObj = new
                {
                    Pagare = credito.consecutivo,
                    FechaCompromiso = credito.fecha_desembolso.ToString("yyyy-MM-dd"),
                    FechaPrimerPago = credito.fecha_inicio_pagos.ToString("yyyy-MM-dd"),
                    ValorCompromiso = credito.monto_desembolsado,
                    Saldo = credito.monto_desembolsado - credito.monto_capital_pagado,
                    Plazo = credito.plazo,
                    ValorCuota = amountApproved,
                    PuntoVenta = oficina.descripcion,
                    Marca = marca.nombre,
                    Empresa = empresa.nombre,
                    Estado = estado.descripcion
                };

                listCreditos.Add(dataObj);
            }

            dataObj = new
            {
                status = "success",
                data = listCreditos,
                code = codeOk,
                mensaje = messageOk
            };

        } else
        {
            hasError = true;
            messageError = infoClient.Message;
            dataObj = new
            {
                status = "error",
                data = new object(),
                code = codeError,
                mensaje = messageError
            };
        }
    }
}
```

- **Operación consultar pagos**

Para este proceso se establece el valor “consultarprestamos” que será enviado de manera encriptada en el mismo json descrito anteriormente. Donde el contenido del atributo “datos” corresponde a una encriptación del siguiente objeto:

```
{
“operacion”: “consultarpagos”,
“identificacion”: “16736984”,
“usuario”: “xxx”,
“password”: “xxx”
}
```

La operación “consultarpagos” tendrá como objetivo retornar los datos de un cliente en concreto de los pagos y las notas de refinanciación que tiene a la fecha históricamente junto a la información que con este se relacione, como el consecutivo, la oficina por donde fue hecho el pago, la fecha y su valor.

```
else if (dataCheckoutDto.Operacion.ToLower() == "consultarpagos")
{
    var json = "{ }";
    var oficina = JsonConvert.DeserializeObject<tbl_oficinas>(json);
    var marca = JsonConvert.DeserializeObject<tbl_marcas>(json);
    var empresa = JsonConvert.DeserializeObject<tbl_empresas>(json);

    using (var dbContext = new BaseContext())
    {
        var idSucursal = int.Parse(sucursal);
        var infoSucursal = dbContext.tbl_oficinas.FirstOrDefault(x => x.id_oficina == idSucursal);
        var infoClient = desembolso.TraerInformacionCliente(dataCheckoutDto.Identificacion, infoSucursal.codigo_interno, false);

        if (infoClient.Data != null)
        {
            DatosDesembolsoDTO datosDesembolsoDTO = JsonConvert.DeserializeObject<DatosDesembolsoDTO>(infoClient.Data);

            var cliente = dbContext.tbl_clientes.FirstOrDefault(x => x.numero_identificacion == datosDesembolsoDTO.NumeroCedula);

            var documentoClasificacionRC = dbContext.tbl_documentos_clasificacion.Select(c => c.fk_id_doc_recibo_caja).ToList();
            var documentoClasificacionNCRF = dbContext.tbl_documentos_clasificacion.Select(c => c.fk_id_doc_nota_credito_refinanciacio

            var comprobantes = dbContext.tbl_comprobantes.Where(x => x.fk_id_cliente == cliente.id_clientes && !(bool)x.anulado && (do

            List<dynamic> listPagos = new List<dynamic>();

            foreach (var comprobante in comprobantes)
            {

                if (comprobante.fk_id_oficina != null)
                {
                    oficina = dbContext.tbl_oficinas.Where(x => x.id_oficina == comprobante.fk_id_oficina).First();
                    marca = dbContext.tbl_marcas.Where(x => x.id_marca == oficina.fk_id_marca).First();
                }
                if (comprobante.fk_id_empresa != null)
                {
                    empresa = dbContext.tbl_empresas.Where(x => x.id_empresa == comprobante.fk_id_empresa).First();
                }

                dataObj = new
                {
                    Documento = comprobante.fk_id_documento_transaccional,
                    Numero = comprobante.consecutivo,
                    FechaPago = comprobante.fecha.ToString("yyyy-MM-dd"),
                    Valor = comprobante.valor_total,
                    PuntoVenta = oficina.descripcion ?? "",
                    Marca = marca.nombre ?? "",
                    Empresa = empresa.nombre ?? ""
                };
                listPagos.Add(dataObj);
            }

            dataObj = new
            {
                status = "success",
                data = listPagos,
                code = codeOk,
                mensaje = messageOk
            };

        } else
        {
            hasError = true;
            messageError = infoClient.Message;
            dataObj = new
            {
                status = "error",
                data = new object(),
                code = codeError,
                mensaje = messageError
            };
        }
    }
}
```

Otros procesos

A. Validar el usuario

```
public bool ValidateUser(string username, string password)
{
    var pass = "";

    try
    {
        using (var dbContext = new BaseContext())
        {
            var result = dbContext.tbl_usuarios.FirstOrDefault(u => u.estado && u.login.ToUpper() == username.ToUpper());
            if (result != null)
            {
                if (result.cambio_clave != null) pass = SecurityUtil.Sha256(password);
                else pass = SecurityUtil.Md5(password);

                return result.password == pass;

            }
            return false;
        }
    }
    catch(Exception ex)
    {
        return false;
    }
}
```

Método que valida si el usuario y la contraseña enviada en la petición después de ser descifrada están registrados como usuarios en el sistema para poder acceder a la información en credi10.

B. **Encriptar**

```
public string Encrypt(string plainText)
{
    if (!string.IsNullOrEmpty(plainText))
    {
        try
        {
            using (Aes aesAlgorithm = Aes.Create())
            {
                string password = GetSha256Hash(Encoding.UTF8.GetBytes(SecretKey));
                byte[] passwordBytes = Encoding.UTF8.GetBytes(password);
                byte[] newPasswordBytes = new byte[32];
                Array.Copy(passwordBytes, 0, newPasswordBytes, 0, 32);
                aesAlgorithm.Key = newPasswordBytes;

                string IV = GetSha256Hash(Encoding.UTF8.GetBytes(SecretIV));
                byte[] ivBytes = Encoding.UTF8.GetBytes(IV);
                byte[] newIVBytes = new byte[16];
                Array.Copy(ivBytes, 0, newIVBytes, 0, 16);
                aesAlgorithm.IV = newIVBytes;

                // Create encryptor object
                ICryptoTransform encryptor = aesAlgorithm.CreateEncryptor();

                byte[] encryptedData;

                //Encryption will be done in a memory stream through a CryptoStream object
                using (MemoryStream ms = new MemoryStream())
                {
                    using (CryptoStream cs = new CryptoStream(ms, encryptor, CryptoStreamMode.Write))
                    {
                        using (StreamWriter sw = new StreamWriter(cs))
                        {
                            sw.Write(plainText);
                        }
                        encryptedData = ms.ToArray();
                    }
                }

                return Convert.ToBase64String(encryptedData);
            }
        }
        catch (Exception ex)
        {
            Console.WriteLine("Error " + ex.Message);
            return plainText;
        }
    }
    return plainText;
}
```

(variable local) Aes aesAlgorithm

Método usado para encriptar información con AES (The Advanced Encryption Standard).

C. **Desencriptar**

```
public string Decrypt(string encryptedData)
{
    if (!string.IsNullOrEmpty(encryptedData))
    {
        try
        {
            using (Aes aesAlgorithm = Aes.Create())
            {
                string password = GetSha256Hash(Encoding.UTF8.GetBytes(SecretKey));
                byte[] passwordBytes = Encoding.UTF8.GetBytes(password);
                byte[] newPasswordBytes = new byte[32];
                Array.Copy(passwordBytes, 0, newPasswordBytes, 0, 32);
                aesAlgorithm.Key = newPasswordBytes;

                string IV = GetSha256Hash(Encoding.UTF8.GetBytes(SecretIV));
                byte[] ivBytes = Encoding.UTF8.GetBytes(IV);
                byte[] newIVBytes = new byte[16];
                Array.Copy(ivBytes, 0, newIVBytes, 0, 16);
                aesAlgorithm.IV = newIVBytes;

                // Create decryptor object
                ICryptoTransform decryptor = aesAlgorithm.CreateDecryptor();

                byte[] cipher = Convert.FromBase64String(encryptedData);

                //Decryption will be done in a memory stream through a CryptoStream object
                using (MemoryStream ms = new MemoryStream(cipher))
                {
                    using (CryptoStream cs = new CryptoStream(ms, decryptor, CryptoStreamMode.Read))
                    {
                        using (StreamReader sr = new StreamReader(cs))
                        {
                            return sr.ReadToEnd();
                        }
                    }
                }
            }
        }
        catch (Exception ex)
        {
            Console.WriteLine("Error " + ex.Message);
            return encryptedData;
        }
    }
    return encryptedData;
}
```

Método usado para desencriptar una cadena de string que este encriptado con AES (The Advanced Encryption Standard)

D. Consultar pago minimo y total

Método que retorna el pago minimo, el pago total y el próximo pago que tendrá un cliente en credi10 y serán los valores que el usuario podrá ver en pantalla para continuar con el proceso por el link de recaudo. Este método es explicado en el documento Documentacion GenerarPagos.docx

E. Traer información del cliente

Método que consulta toda la información relacionada de un cliente en específico.

1.3.4 Documentación de Arquitectura

Lógica de los demás procesos en el API de servicios:

- LinkDePagos.docx
- BotonEcommerce.docx

1.3.5 Log

En este proceso no es generado ningún log de auditoría.

1.3.6 Lista de Programas, archivos, paquetes

- **sp_cupo_disponible:**
  Procedimiento almacenado que retorna el valor del cupo aprobado y el monto disponible que tiene en credi10 de un cliente especifico.

1.3.7 Lista de modificaciones hacia la base de datos

Script 1. Creación de tablas: Para crear el esquema completo de la base de datos de la aplicación.

Creacion_BD_C10_Credi10Net.sql

1.4 Observaciones y/o Comentarios

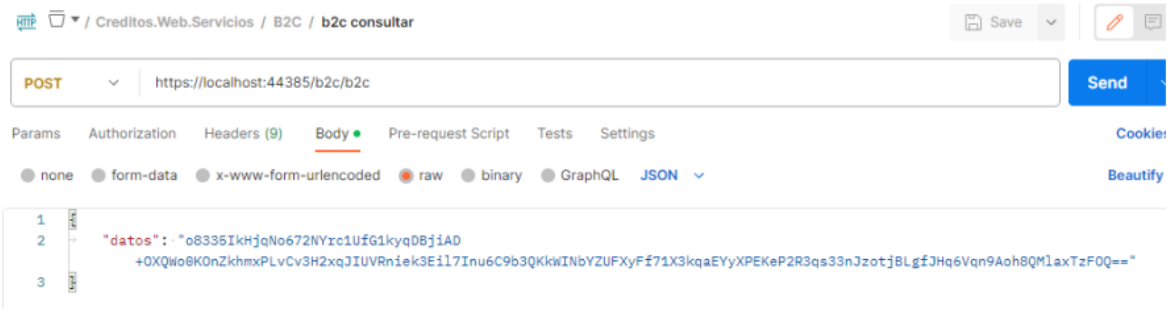
En el archivo coleccionPostman.json se encontrará la colección que podrá ser exportada a postman con todas las consultas que podrán ser realizadas al API de servicios.

2. CASOS DE PRUEBAS UNITARIAS

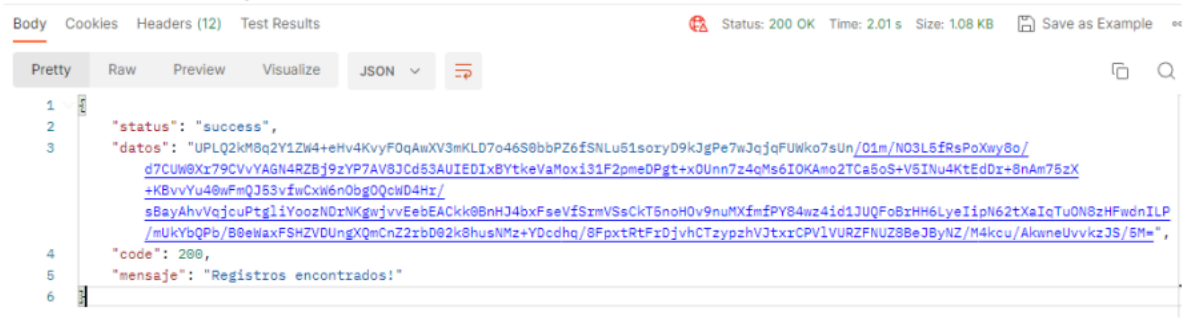
A continuación, se demuestra las pruebas funcionales hechas en el ambiente de pruebas.

1. Para llevar a cabo las pruebas de consulta de información del cliente de B2C, se ejecuta el request en Postman y se obtiene la siguiente respuesta satisfactoria:

- Envió de consulta de información de cliente B2C

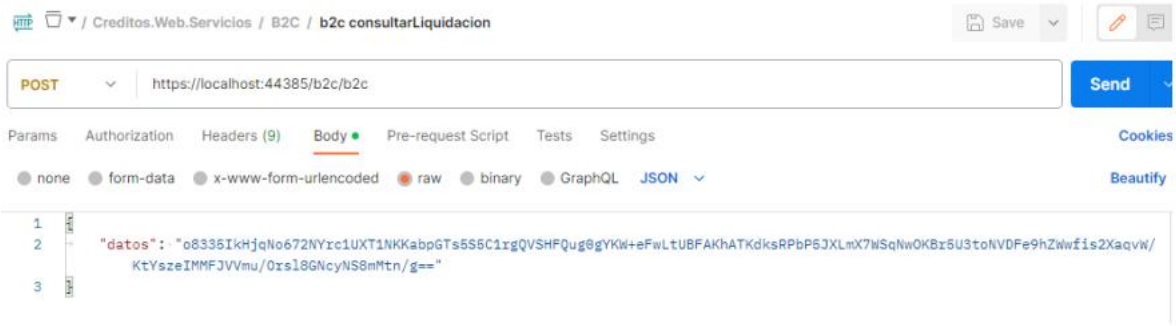


- Respuesta de consulta de información de cliente B2C

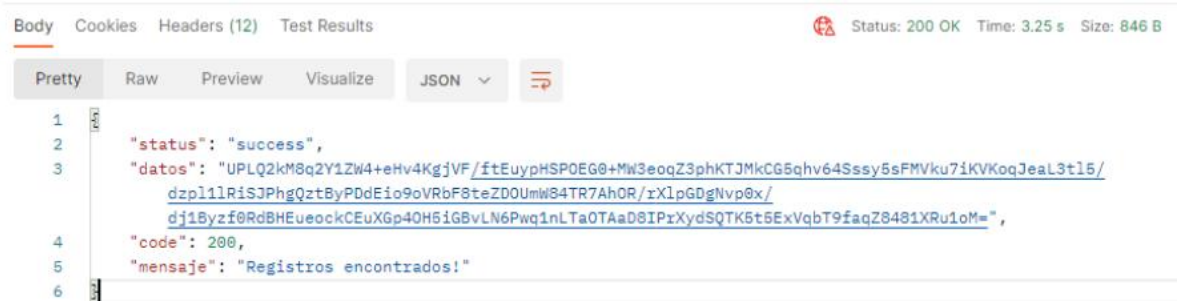


2. Para llevar a cabo las pruebas de consulta de información del cliente de B2C, se ejecuta el request en Postman y se obtiene la siguiente respuesta satisfactoria:

- Envió de consulta de información de la liquidación del cliente.



- Respuesta de consulta de información de la liquidación del cliente.

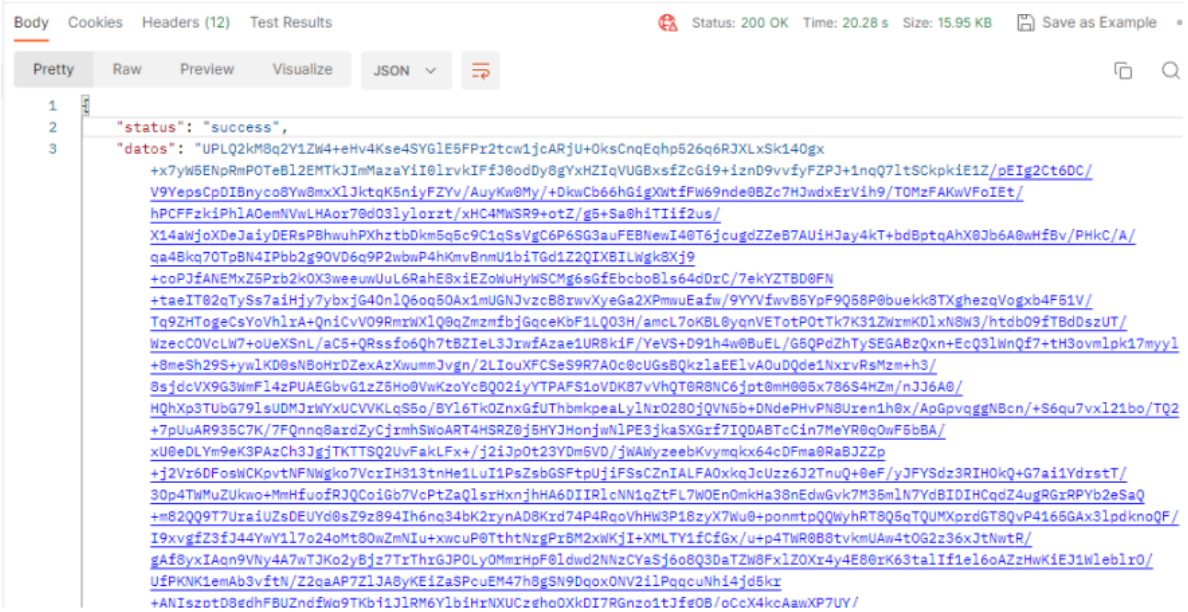


3. Para llevar a cabo las pruebas de consulta de préstamos del cliente de B2C, se ejecuta el request en Postman y se obtiene la siguiente respuesta satisfactoria:

- Envió de consulta de información de préstamos del cliente.

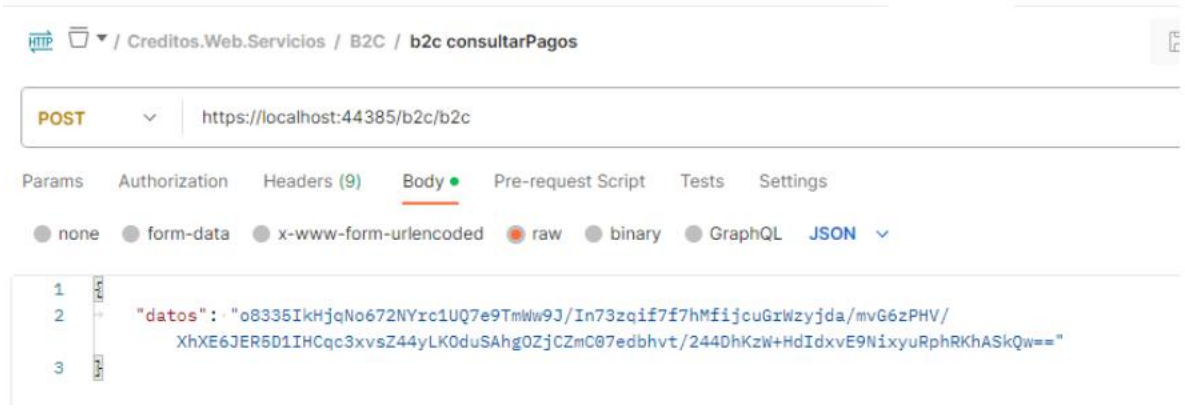


- Respuesta de consulta de información de préstamos del cliente.

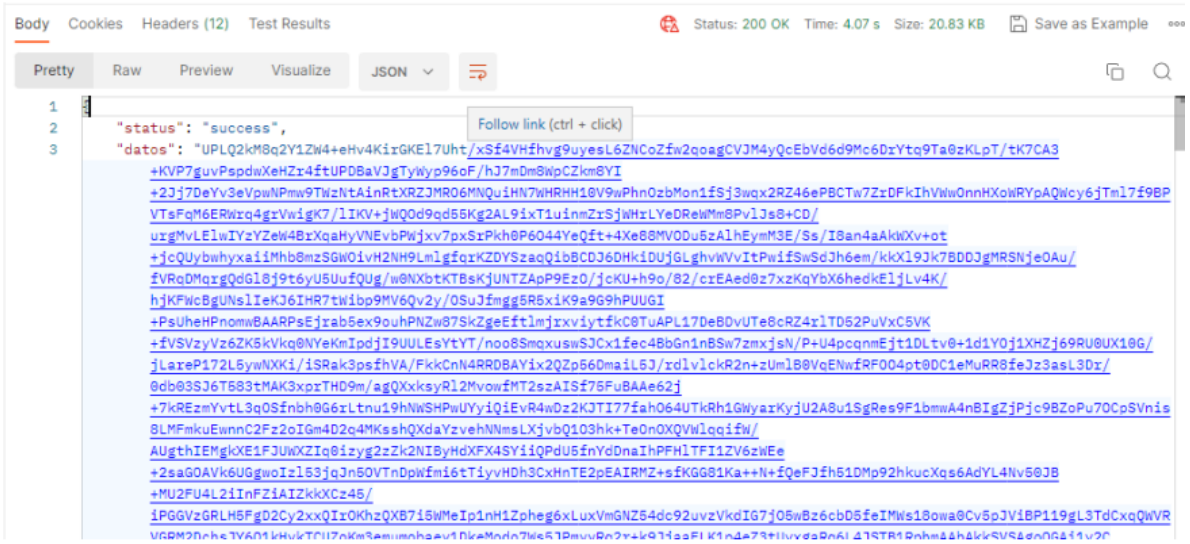


4. Para llevar a cabo las pruebas de consulta de pagos del cliente de B2C, se ejecuta el request en Postman y se obtiene la siguiente respuesta satisfactoria:

- Envió de consulta de información de pagos del cliente.



- Respuesta de consulta de información de pagos del cliente.



ID	CASO DE PRUEBA	RESULTADO ESPERADO	APROBADO
1	Ejecución de API consulta B2C.	Información de consulta de cliente B2C con encriptación.	SI
2	Ejecución de API consulta liquidación B2C.	Información de consulta de liquidación B2C con encriptación.	SI
3	Ejecución de API consulta prestamos B2C.	Información de consulta de préstamos B2C con encriptación.	SI

4	Ejecución de API consulta pagos B2C.	Información de consulta de pagos B2C con encriptación.	SI